



Software Requirements Specification

Green Fork – Restaurant App MVP (Frontend Only, React Native)

SUBMITTED BY

Muhammad Wasim

Registration No. **9513**

Mobile Application Development • Assignment 1 • Fall 2026

Document	SRS – Question 1 (version 1.0, 29 September 2026)
Technology	React Native 0.86 • Expo SDK 57 • React Navigation 7 • AsyncStorage
Repository	github.com/wasikayani/restaurant-app-mvp
Demo video	youtu.be/FckzcB1fk6k

1. Introduction	2
2. User Roles • 3. User Stories	3
4. Functional Requirements • 5. Non-Functional Requirements	4
6. Mock Data Model • UML Diagrams	6
7. Screen Specifications (7.1 – 7.8)	9

1. Introduction

1.1 Purpose

This SRS defines the requirements of **Green Fork**, a continental and fast-food restaurant app built as a frontend-only MVP with React Native and Expo. It is the reference for evaluation, the viva and future development.

1.2 Scope

Customers browse a menu, search and filter dishes, build a cart with promo codes, place Dine-in or Takeaway orders, track them live and reserve tables. Managers handle incoming orders, approve reservations and edit the menu from a dashboard. The app runs on a phone through Expo Go.

In scope

- Mock login / sign-up with Customer and Manager roles
- Menu (19 dishes, 4 categories), search, sort, favourites
- Cart, promo codes, order summary with tax
- Dine-in / Takeaway orders with live tracking
- Table reservations with availability checks
- Manager dashboard, dark mode, AsyncStorage saving

Out of scope

- Backend server, database or external API
- Real payments and home delivery
- Real authentication security (hashing, tokens)
- Push notifications, SMS and email
- Sync between different phones, multiple branches
- App Store / Play Store publishing

1.3 Definitions

Term	Definition
MVP	Minimum Viable Product – the smallest version that shows the full idea working.
Expo Go	Phone app that runs the project by scanning a QR code from the development server.
Hook	React function (e.g. <code>useState</code>) that adds state or behaviour to a function component.
Custom hook	Developer-written hook starting with <code>use</code> , e.g. <code>useForm</code> , <code>useReservation</code> .
Context	Shares data (user, theme, cart, orders) with any component without prop drilling.
Reducer	Pure function <code>(state, action) → newState</code> used with <code>useReducer</code> .
AsyncStorage	Key-value storage on the phone that keeps data after the app closes.
Mock data	Realistic fake data stored in JavaScript files instead of a database.
Debounce	Waiting 400 ms after the user stops typing before searching.
Dine-in / Takeaway	Eat at a chosen table, or collect the order at a chosen pickup time.

2. User Roles

Role	Description	Permissions	Demo account
Customer	Guest who orders food or books a table.	Browse, search, favourite; cart and promo codes; place, track and cancel own orders; reserve and cancel tables; dark mode.	customer@greenfork.pk / customer123
Manager	Staff member who runs daily operations.	View the menu plus the Dashboard: update or cancel orders, accept / decline reservations, add dishes, edit prices, toggle availability.	manager@greenfork.pk / manager123

3. User Stories

Stories US-01 to US-06 come from a **customer self-interview**: I asked myself, as a regular fast-food customer, what annoys me when ordering (queues, unknown prices and availability), how I choose food, how I customise orders, whether I use discounts, what I want to know after ordering, and how I book family tables.

ID	User story	Source
US-01	As a customer , I want to browse the menu with photos, prices and availability, so that I can decide before I reach the counter.	Self-interview
US-02	As a customer , I want to filter by category, search by name and sort by price, so that I find a dish in seconds.	Self-interview
US-03	As a customer , I want to change quantities and add a note to each item, so that my order is exactly right.	Self-interview
US-04	As a customer , I want to apply a promo code and see the full bill with tax, so that I know what I will pay.	Self-interview
US-05	As a customer , I want to choose Dine-in or Takeaway and follow my order live, so that I know when my food is ready.	Self-interview
US-06	As a customer , I want to see free time slots for my party size and book a table, so that my family has a place.	Self-interview
US-07	As a customer , I want to mark favourite dishes, so that I can find my usual order quickly.	Customer
US-08	As a user , I want a dark mode, so that the app is comfortable at night.	Both
US-09	As a manager , I want to move orders through Preparing, Ready and Served, so that the kitchen and customers stay in sync.	Manager
US-10	As a manager , I want to accept or decline pending reservations, so that I control how full the restaurant gets.	Manager
US-11	As a manager , I want to add dishes, edit prices and switch dishes off, so that customers only order what we can make.	Manager
US-12	As a manager , I want an overview of active orders, bookings and revenue, so that I know where attention is needed.	Manager

4. Functional Requirements

ID	Requirement	Q
Authentication		
FR-01	The system shall let a visitor switch between Login and Sign Up on one screen.	Q3
FR-02	The system shall validate email format and a password of 8+ characters with a digit; sign-up shall also require full name, matching confirm password and a role.	Q3
FR-03	The system shall show a spinner for 1 second while logging in and an Alert when credentials are wrong.	Q3
FR-04	The system shall open Menu for customers and Dashboard for managers, and show the Dashboard tab only to managers.	Q3, Q6
FR-05	The system shall log the user out after confirmation and reset navigation to Login.	Q6
Menu		
FR-06	The system shall load the menu through a simulated 1.5 s request with a loading state, and show an error with a Retry button on failure.	Q4
FR-07	The system shall filter dishes by category and show the item count in the header.	Q4
FR-08	The system shall mark Daily Specials and disable Add for unavailable dishes.	Q4
FR-09	The system shall support pull-to-refresh, sorting (price, name) and favourites.	Q4, Q8
Search		
FR-10	The system shall search dishes 400 ms after the user stops typing.	Q5, Q9
FR-11	The system shall suggest the last 5 searches and show an empty state when nothing matches.	Q5
FR-12	The system shall show a Back-to-top button after scrolling 300 px.	Q5
Cart		
FR-13	The system shall add dishes to the cart, increase / decrease quantities, remove an item at 0 and show a cart badge.	Q7
FR-14	The system shall save a note for each cart item.	Q7
FR-15	The system shall accept promo codes WELCOME10 (10%) and FEAST20 (20%) and reject invalid codes.	Q7
FR-16	The system shall calculate subtotal, discount, 5% service charge, 15% tax and grand total.	Q8
Reservation		
FR-17	The system shall let the customer pick a date, party size (1-12), hourly slot (12:00-22:00) and a fitting table.	Q9
FR-18	The system shall disable slots with no free table or less than 1 hour away.	Q9
FR-19	The system shall validate the date, name and phone (03XX-XXXXXXX) and confirm in a modal before saving.	Q9
FR-20	The system shall list the customer's reservations and allow cancelling after an Alert.	Q9
Orders		
FR-21	The system shall require Dine-in with a table or Takeaway with a pickup time before placing an order.	Q10
FR-22	The system shall create an order (id, items, total, type, status Pending, timestamp), clear the cart and open tracking.	Q10

FR-23	The system shall move the order to Preparing (10 s), Ready (20 s) and Served (30 s) with step progress and elapsed time.	Q10
FR-24	The system shall let the customer cancel an order while it is Pending or Preparing.	Q10
Manager Dashboard		
FR-25	The system shall let the manager move orders to the next status or cancel them.	Q10
FR-26	The system shall let the manager accept or decline pending reservations.	Q10
FR-27	The system shall let the manager add dishes, edit prices and toggle availability, shown to customers immediately.	Q10
FR-28	The system shall save orders, reservations and menu edits with AsyncStorage and show a loading screen until loaded.	Q10

5. Non-Functional Requirements

Category	Requirement
Usability	A first-time user can place an order in under 2 minutes; errors appear next to their cause in plain language; clear icons, labels and confirmation popups.
Performance	Only the changed menu card re-renders (<code>React.memo</code> + <code>useCallback</code> : 19 → 1 renders); lists and totals are memoised with <code>useMemo</code> ; usable content within 3 s of opening.
Responsiveness	Layouts adapt to all phone sizes with Flexbox and safe areas; the UI never freezes (all waits are asynchronous, buttons show spinners).
Maintainability	Code is split into components, screens, context, reducers, hooks, data, navigation, theme and utils; business rules are pure functions with 40 Jest tests; all colours come from one theme file.
No-backend data handling	All data is mock data with simulated delays; orders, reservations and menu edits are saved in AsyncStorage; if storage fails the app keeps working with in-memory data.

6. Mock Data Model

Entity	Fields (type)	Source
Users	id, fullName, email, password (string), role ("customer" "manager")	data/users.js
Categories	id, label, icon (string) – Starters, Mains, Desserts, Drinks	data/menu.js
MenuItems	id, name, description, image, category (string), price, rating, prepTime (number), isSpecial, isAvailable (boolean)	data/menu.js · MenuContext
Tables	id, area (string), number, seats (number) – 8 tables, 2 to 12 seats	data/tables.js
Reservations	id, userId, date, time, tableId, name, phone, note (string), partySize, tableNumber (number), status (Pending Confirmed Declined Cancelled)	data/reservations.js · ReservationContext
Orders	id, userId, customerName (string), items (CartItem[]), subtotal, discount, serviceCharge, salesTax, total, timestamp (number), type (Dine-in Takeaway), tableId / pickupTime, status (Pending Preparing Ready Served Cancelled)	OrdersContext (runtime)

UML Diagrams

The five diagrams of Question 2. Full-size PNG files are in [A1/UML](#).

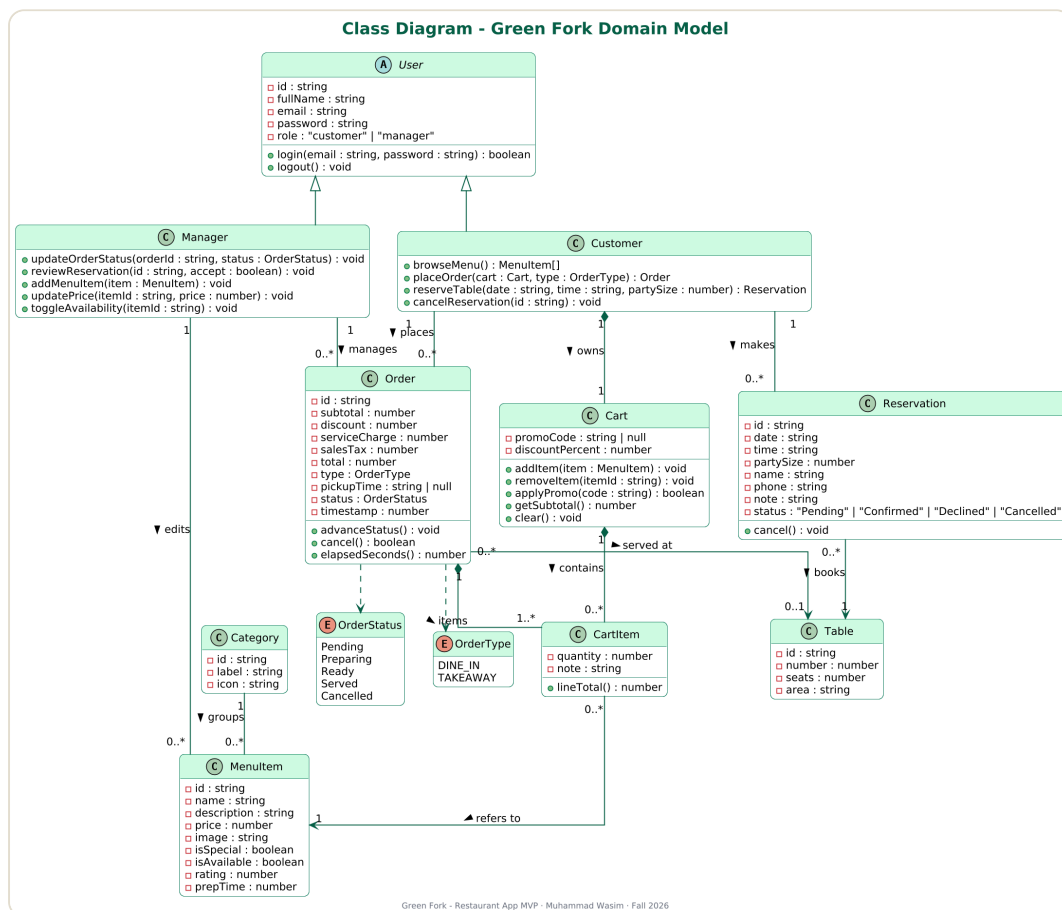


Figure 1 – Class diagram: User → Customer / Manager inheritance, Cart, CartItem, MenuItem, Category, Order, Reservation, Table with multiplicities.

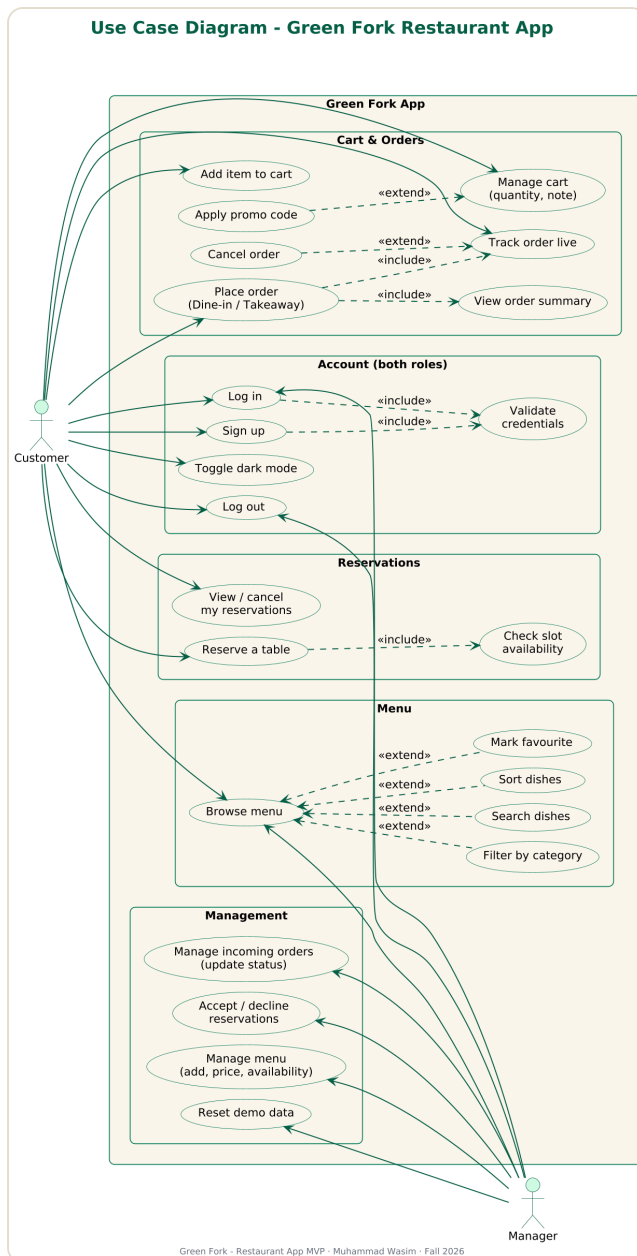


Figure 2 – Use case diagram (23 use cases with include / extend).

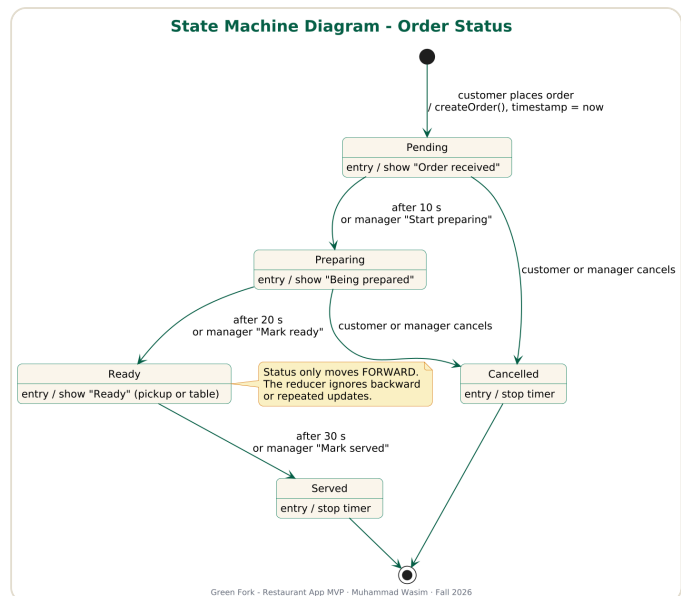


Figure 3 – State machine: Pending → Preparing → Ready → Served, or Cancelled.

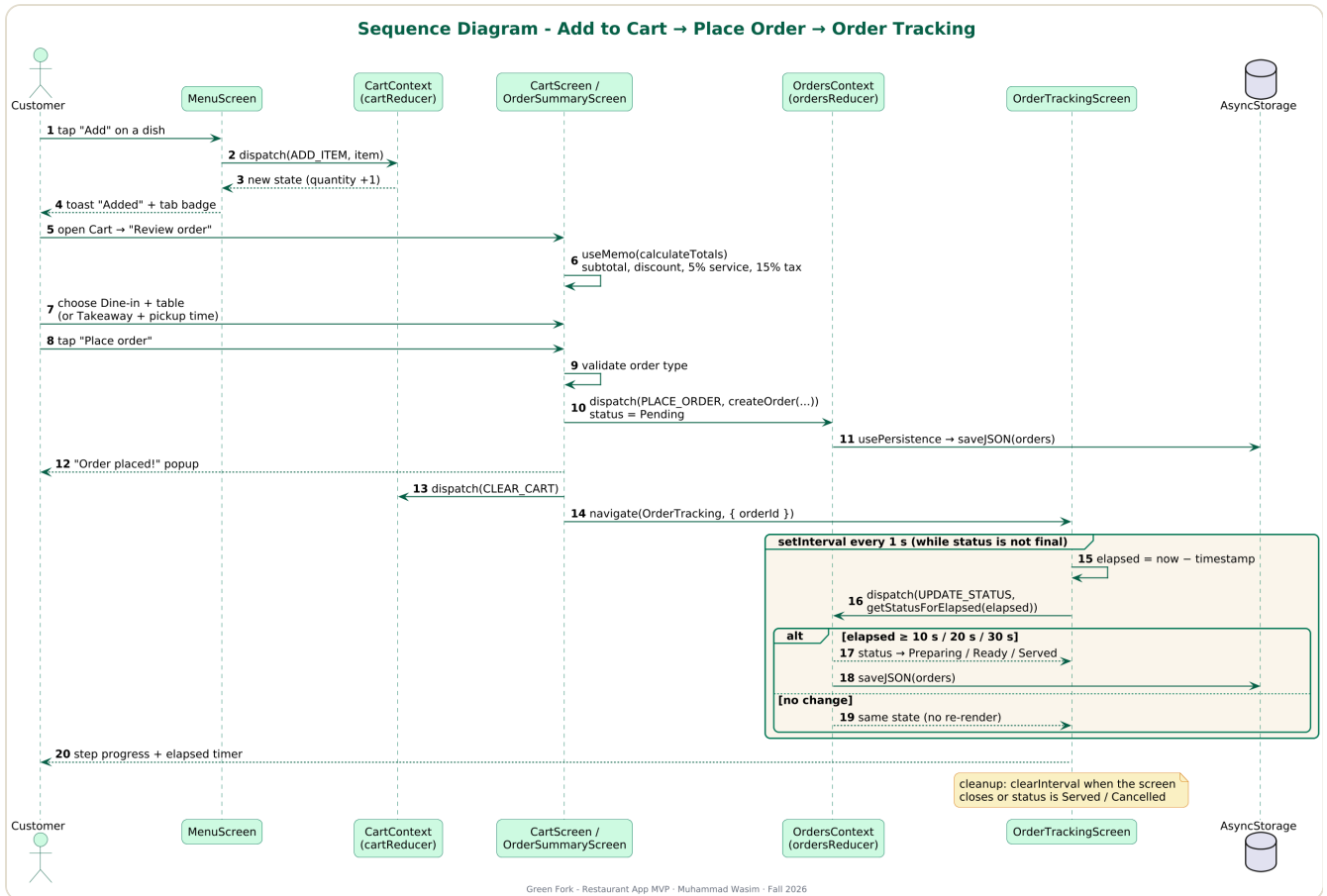


Figure 4 – Sequence diagram: Add to Cart → Place Order → Order Tracking.

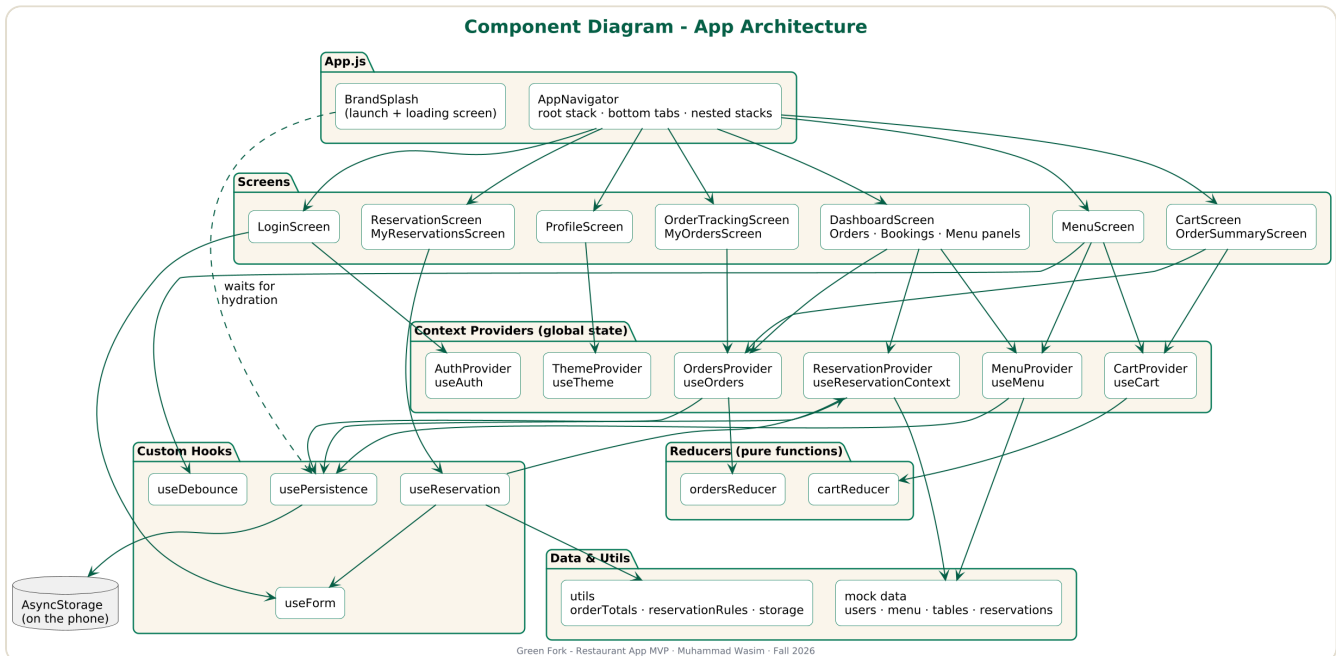
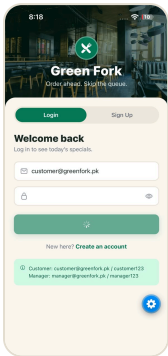


Figure 5 – Component diagram: screens, context providers, reducers, custom hooks, data and AsyncStorage.

7. Screen Specifications



7.1 Login / Sign Up LoginScreen.js

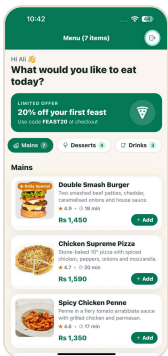
Purpose: log in or create an account and open the right area for the role.

UI: Login / Sign Up switch, email and password (show/hide), full name, confirm password, role cards, spinner button, success popup.

Navigation: Customer → Menu tab, Manager → Dashboard tab.

Data: mock users (find, check email, add); logged-in user saved in AuthContext.

Hooks and why: `useState` for mode, show-password, isSubmitting; custom `useForm` for values and errors that clear on edit; `useAuth` to store the user globally.



7.2 Menu MenuScreen.js, MenuItemCard.js

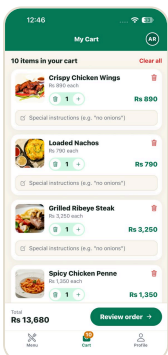
Purpose: show the menu and help the customer find and add dishes.

UI: search with recent searches, category chips, sort pills, dish cards (photo, special badge, price, heart, Add / N/A), skeleton, error + Retry, empty state, pull-to-refresh, Back to top, toast.

Navigation: Menu tab; toast opens Cart.

Data: shared menu (MenuContext) loaded with a simulated fetch; cart dispatch.

Hooks and why: `useEffect` loads once with cleanup; `useRef` for input focus, list scroll and render counter without re-renders; `useMemo` for the filtered/sorted list; `useCallback` + `React.memo` so only the tapped card re-renders; `useDebounce` for search.



7.3 Cart CartScreen.js

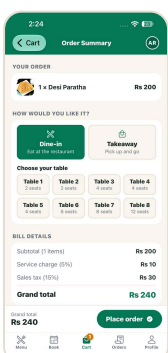
Purpose: review and adjust the order.

UI: item rows with + / - stepper, remove, note; promo code input with error; mini bill; Review order; empty state; tab badge.

Navigation: Cart tab → Order Summary.

Data: `{ items, promoCode, discountPercent }` from CartContext.

Hooks and why: `useReducer` (via `useCart`) keeps 8 related cart actions in one pure, tested reducer; `useState` for the promo text box.



7.4 Order Summary OrderSummaryScreen.js

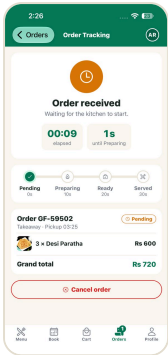
Purpose: show the final bill and place the order.

UI: items, Dine-in / Takeaway cards, table or pickup-time chips, bill (subtotal, discount, 5% service, 15% tax, total), Place order, success popup.

Navigation: after placing → Orders tab › Order Tracking.

Data: cart items + `calculateTotals`; new order sent to OrdersContext.

Hooks and why: `useMemo` recalculates totals only when items or discount change; `useState` for the order choices; `useEffect` timer with cleanup before navigating.



7.5 Order Tracking and My Orders OrderTrackingScreen.js, MyOrdersScreen.js

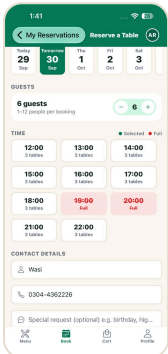
Purpose: follow an order live and see order history.

UI: status icon and title, elapsed timer, time to next step, 4-step progress bar, order details, Cancel; My Orders list (in progress / past).

Navigation: Orders tab: My Orders → Order Tracking.

Data: order from OrdersContext; status updates dispatched to `ordersReducer`.

Hooks and why: `useEffect` + `setInterval` ticks every second and advances the status (10 s / 20 s / 30 s), `clearInterval` on cleanup; `useState` for elapsed seconds.



7.6 Reservation and My Reservations ReservationScreen.js, MyReservationsScreen.js

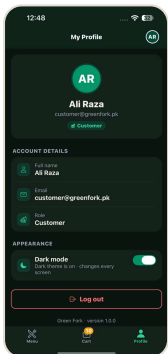
Purpose: book a table only when one is free, and manage bookings.

UI: date chips, guest stepper, slot grid (Full slots crossed out), table chips, contact inputs, confirmation modal; list with status and Cancel.

Navigation: Book tab: Reservation → My Reservations.

Data: mockTables and all reservations (ReservationContext).

Hooks and why: custom `useReservation` holds all booking logic so the screen is UI only; it uses `useForm` for validation and `useMemo` for free slots and tables.



7.7 Profile and Account Menu ProfileScreen.js, AccountMenu.js

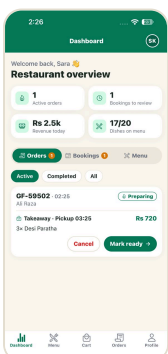
Purpose: show the logged-in user, switch theme and log out.

UI: avatar, name, email, role badge, dark mode switch, Reset demo data (manager), Log out; header avatar opens the account menu.

Navigation: Profile tab; account menu links to Profile, My Reservations and My Orders.

Data: user from AuthContext, theme from ThemeContext.

Hooks and why: `useContext` (`useAuth` , `useTheme`) because user and theme are needed on almost every screen – no prop drilling.



7.8 Manager Dashboard DashboardScreen.js + dashboard panels

Purpose: one place for the manager to run the restaurant.

UI: 4 live cards (active orders, bookings to review, revenue, dishes); tabs Orders (next status / cancel), Bookings (accept / decline), Menu (add dish, edit price, availability switch).

Navigation: Dashboard tab, only for `role === 'manager'`.

Data: OrdersContext, ReservationContext, MenuContext – shared with customers and saved in AsyncStorage.

Hooks and why: `useContext` to read and update shared data; `useMemo` for statistics and filtered lists; `useState` / `useForm` for tabs, filters and the add-dish form.